

TRACCIA di DOCUMENTAZIONE

1. Testo :

Il progetto consiste nella realizzazione di un **chatbot di assistenza clienti** per Utixo, in grado di rispondere automaticamente alle domande frequenti degli utenti su servizi come hosting, email, PEC, DNS, SSL, backup e cloud. Le FAQ sono memorizzate nel database e vengono utilizzate dal chatbot per fornire risposte rapide.

Le finalità del progetto sono:

- offrire assistenza immediata agli utenti;
- ridurre il carico del supporto umano;
- rendere il servizio disponibile 24/7;
- registrare le richieste per migliorare il sistema;
- permettere agli amministratori di aggiornare le FAQ.

Gli ambiti di applicazione sono il **customer care**, l'**assistenza tecnica di primo livello** e l'**automazione del supporto clienti**.

2. Analisi del contesto:

Molte aziende usano chatbot per gestire richieste ripetitive e migliorare i tempi di risposta. Nel caso reale, Utixo dispone già di una knowledgebase di FAQ tecniche e informative. Il sistema salva anche utenti, conversazioni e log, quindi il chatbot non è solo un sistema di risposta, ma una piattaforma completa di assistenza e monitoraggio.

a. Stakeholders

• Analisi dei principali stakeholder

- clienti Utixo
- Roberto Mazzei (Utixo's CEO)
- Dipendenti dell'azienda

• Matrice power/interest

- **alto potere / alto interesse:** CEO
- **basso potere / alto interesse:** Clienti
- **basso potere / alto interesse:** Dipendenti

• Ipotesi di gestione del livello di coinvolgimento nel progetto

- CEO e responsabili: aggiornamento costante
- Dipendenti: validazione delle FAQ

- utenti: test e analisi

3. Ipotesi :

a. prerequisiti/dati di fatto

- esiste un database “**chatbot**”;
- l'applicazione usa **Python + Flask**;
- il database è **MySQL/MariaDB**;
- il chatbot usa **TF-IDF e cosine similarity** per confrontare le richieste con le FAQ;
- il sistema gestisce utenti, conversazioni, log e FAQ.

b. exclusions

- nessuna integrazione con LLM esterni;
- nessuna app mobile nativa;
- nessun passaggio automatico a operatore umano;
- nessun sistema NLP avanzato oltre al matching testuale.

4. Obiettivi:

Gli obiettivi del progetto sono:

- fornire risposte automatiche rapide;
- creare un'interfaccia semplice e intuitiva;
- salvare lo storico delle conversazioni;
- registrare i log delle richieste;
- permettere agli admin di gestire le FAQ;
- migliorare la qualità del supporto clienti.

5. Target :

Il target principale è costituito dai **clienti Utixo**, che necessitano di assistenza veloce, semplice e sempre disponibile.

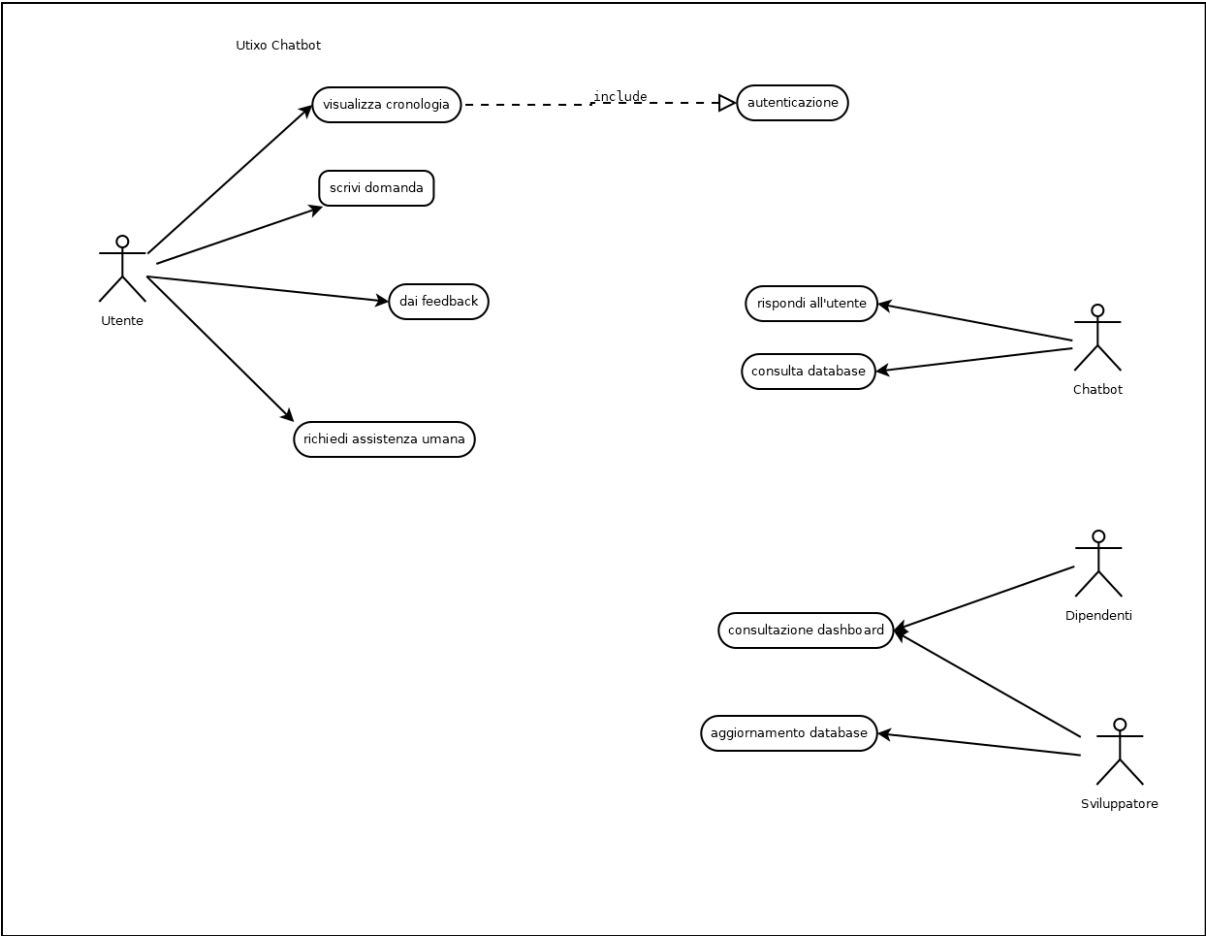
Il target secondario è costituito da **amministratori e personale di supporto**, che devono monitorare il sistema e aggiornare le FAQ.

6. UML: diagramma e user stories

User stories

ID	Epic	User Story	Priorità	Tempo
1	Raccolta esigenze	Come Product Owner voglio raccogliere le esigenze dei clienti e del CEO, così da definire gli obiettivi del chatbot	Alta	3 giorni
2	Creazione Project Charter	Come Product Owner voglio identificare gli stakeholder e definire tutti gli scope del progetto	Media	1 giorno
3	Gestione privacy e policy	Come cliente voglio un chatbot affidabile e sicuro dal punto di vista della privacy	Alta	3 giorni
4	Interfaccia chat	Come cliente voglio un'interfaccia semplice e intuitiva	Alta	3 giorni
5	Riconoscimento richieste	Come chatbot voglio comprendere le richieste degli utenti, così da fornire risposte pertinenti	Alta	10 giorni
6	Risposte predefinite	Come cliente voglio ricevere risposte immediate e chiare per problemi comuni, così da risolverli senza attendere operatori	Alta	3 giorni
7	Escalation a operatore	Come utente voglio poter parlare con un operatore umano se il chatbot non risolve il mio problema, così da ricevere assistenza completa	Media	4 giorni
8	Logging e sicurezza	Come sviluppatore voglio registrare conversazioni e metadata in sicurezza, così da analizzare e migliorare il chatbot	Alta	2 giorni

9	Dashboard analytics	Come team support voglio poter vedere metriche di utilizzo del chatbot, così da valutare performance e soddisfazione	Bassa	2 giorni
10	Feedback utenti	Come cliente voglio poter lasciare feedback sulla conversazione, così da contribuire al miglioramento del servizio	Bassa	1 giorno
11	Aggiornamento continuo	Come team di prodotto voglio poter aggiornare regolarmente risposte e flussi, così da mantenere il chatbot aggiornato	Media	1 giorno
12	Test e QA	Come sviluppatore voglio testare il chatbot in scenari reali, così da garantire affidabilità e qualità	Alta	3 giorni



7. Strategia di soluzione :

La soluzione proposta consiste in un **chatbot web client-server** collegato a un database relazionale. L'utente inserisce una domanda nell'interfaccia web; il server Flask elabora il testo, confronta la richiesta con le FAQ presenti nel database tramite **TF-IDF + cosine similarity** e restituisce la risposta più adatta. Se l'utente è autenticato, la conversazione viene salvata e aggiornata. Gli amministratori possono invece inserire, modificare o cancellare le FAQ tramite pannello dedicato.

a. hardware

• infrastruttura di rete

1. schema

Client web → rete Internet → server Flask → database MySQL.

2. apparati

- PC o smartphone client
- router/modem
- switch
- server applicativo
- server database

3. collegamenti

Connessione HTTP/HTTPS tra client e server applicativo; connessione TCP interna tra applicazione Flask e DB MySQL.

4. server

Un server applicativo ospita il chatbot e il pannello admin.

5. db server

Un server MySQL/MariaDB ospita tabelle utenti, FAQ, conversazioni e log.

6. eventuale scalabilità

In futuro si può separare il server web dal DB server, usare backup automatici e bilanciamento del carico.

b. software

• tecnologia utilizzata

1. applicazione web

Applicazione web sviluppata in **Python** con **Flask**, accessibile da browser.

2. applicazione client-server (prodotti, linguaggi, protocolli, servizi di terze parti, API, ...)

- linguaggio: Python
- framework: Flask
- database: MySQL/MariaDB
- librerie: mysql.connector, werkzeug.security, scikit-learn
- protocollo: HTTP/HTTPS
- architettura: client-server.

• base di dati relazionale

1. modello E/R

a. entità

- utenti
- faq
- conversations
- logs

b. attributi: tipi e domini

- utenti: id, nome, password, email, data_creazione
- faq: id, categoria, domanda, risposta1, risposta2, risposta3, data_aggiornamento
- conversations: id, user_id, title, created_at, updated_at
- logs: id, user_id, conversation_id, messaggio_utente, risposta_bot, similarity, faq_id, resolved, data_ora.

c. definizione delle chiavi primarie

Le chiavi primarie sono id in tutte le tabelle. Sono presenti anche chiavi esterne tra conversazioni/utenti e logs/conversations-utenti.

e. regole di lettura

Un utente può avere più conversazioni; una conversazione può contenere più log; una FAQ può essere collegata a più log.

- **base di dati noSQL**

1. Json / XML

Non è prevista una base dati NoSQL. Il formato JSON viene però usato nelle risposte API del chatbot tra front end e back end.

- **class diagram**

Le classi principali possono essere rappresentate logicamente come:

- Utente
- FAQ
- Conversation
- Log
- ChatbotService

La logica applicativa è gestita nel file Flask tramite funzioni di servizio e route.

8. Implementazione :

a. Work Breakdown Structure degli obiettivi del progetto

La WBS può essere sintetizzata in:

1. analisi requisiti
2. progettazione database
3. progettazione chatbot
4. sviluppo front end
5. sviluppo back end
6. implementazione autenticazione
7. gestione FAQ admin
8. test e collaudo
9. documentazione finale

b. Definizione di tempi e risorse per l'attuazione del progetto

Una possibile pianificazione è:

- **Sprint 1:** project charter, stakeholder, obiettivi, raccolta esigenze, flussi chatbot
- **Sprint 2:** interfaccia chat, riconoscimento richieste, risposte predefinite, test e QA
- **Sprint 3:** autenticazione, storico conversazioni, log, ampliamento db
- **Sprint 4:** rifinitura, sicurezza, documentazione e presentazione., area admin

c. front end

- **mockup dell'interfaccia**

L'interfaccia prevede:

- area chat laterale;
- campo di input messaggio;
- pulsante invio;
- eventuale sidebar con cronologia conversazioni;
- login admin separato.

- **linguaggio**

HTML, CSS, JavaScript.

- **ambiente**

Browser web.

- **librerie / oggetti**

Template Flask e componenti HTML standard.

d. base di dati :

- **relazionale: normalizzazione e integrità referenziale**

Il database è relazionale, con tabelle separate per utenti, FAQ, conversazioni e log. La struttura è sufficientemente normalizzata perché evita ridondanze principali e usa chiavi esterne per mantenere l'integrità referenziale.

- **no-sql: descrizione della struttura**

Non utilizzato.

e. back end

- **linguaggio**

Python.

- **ambiente**

Flask su server locale o Linux/Windows server.

- **librerie / oggetti / API**

- Flask
- mysql.connector
- werkzeug.security
- scikit-learn
- dotenv.

9. Distribuzione :

Il prodotto può essere distribuito come **applicazione web pubblica** accessibile tramite browser.

Per gli utenti finali non è necessaria una formazione complessa, perché l'interfaccia è semplice e intuitiva.

Per gli amministratori si prevede una breve formazione sull'uso del pannello FAQ, sul controllo dei log e sull'aggiornamento dei contenuti.

10. Valutazione :

Gli indicatori di valutazione possono essere:

- percentuale di richieste risolte;
- numero di richieste non riconosciute;
- facilità d'uso percepita dagli utenti;
- esito positivo dei test previsti nello sprint QA.

11. Sviluppi :

Gli sviluppi futuri possibili sono:

- integrazione con modelli NLP/LLM più avanzati;
- collegamento a ticketing o operatore umano;
- pannello statistiche più evoluto;
- suggerimenti automatici di nuove FAQ dai log;
- autenticazione più avanzata;
- maggiore scalabilità;
- supporto multilingua;
- versione mobile o responsive migliorata.

12. GDPR :

Il sistema tratta dati personali come username, email, password cifrate e cronologia delle interazioni. Per questo devono essere rispettati i principi del GDPR: minimizzazione dei dati, limitazione delle finalità, sicurezza del trattamento e conservazione controllata. Nel database le password risultano memorizzate in forma hashata, scelta coerente con la protezione delle credenziali.

Le principali misure da adottare sono:

- accesso autenticato per utenti e amministratori;
- cifratura delle password;
- uso di HTTPS;
- controllo degli accessi al database;
- protezione della rete con firewall;
- policy di conservazione dei log;
- limitazione dei privilegi degli amministratori;

Dal punto di vista della sicurezza, i rischi principali sono:

- accessi non autorizzati;
- attacchi alle credenziali;
- SQL injection;
- perdita dei dati;
- uso improprio del pannello admin;
- intercettazione del traffico di rete.